

Calculadora Recursiva de Rutas Combinatorias en Tableros Bidimensionales

Autor: Diego Mauricio Ugas Oliveros – CI-2125

Resumen

El presente trabajo detalla el desarrollo de un motor combinatorio para calcular rutas unidireccionales en matrices espaciales. Se implementó una solución funcional en el lenguaje Zilly+ utilizando algoritmos recursivos sobre grafos, sorteando sintácticamente la ausencia de operadores de igualdad. Para la interacción con el usuario, se construyó una arquitectura encapsulada en C++ que renderiza el entorno bidimensional. El análisis de complejidad demuestra la viabilidad de las funciones de recurrencia espacial para resolver problemas de programación dinámica.

Antecedentes

El problema de calcular trayectorias en una cuadrícula tiene raíces profundas en la combinatoria clásica y la teoría de grafos, ramas de las matemáticas discretas que estudian las propiedades de las estructuras interconectadas. Históricamente, la formalización de rutas en redes comenzó con la resolución del problema de los puentes de Königsberg por Leonhard Euler en 1736, sentando las bases topológicas que hoy utilizamos en las ciencias computacionales.

En el contexto específico de las matrices bidimensionales, el cálculo de caminos unidireccionales (movimientos restringidos exclusivamente hacia el este y el sur) está intrínsecamente ligado al Triángulo de Pascal. Blaise Pascal, en el siglo XVII, organizó de forma tabular los coeficientes binomiales, descubriendo que cada número en su matriz es la suma de los dos números directamente por encima de él. Esta propiedad aditiva es matemáticamente idéntica a la sumatoria de trayectorias en un tablero cuadriculado.

En la computación moderna, este tipo de problemas originó el campo de la Programación Dinámica, introducido por Richard Bellman en la década de 1950. Bellman demostró que los problemas de optimización espacial complejos pueden desglosarse en subproblemas superpuestos más simples. Nuestro proyecto toma este principio fundamental y lo aplica a través de la recursividad pura, explorando cómo lenguajes académicos estrictos (como Zilly+) y lenguajes orientados a objetos (como C++) pueden colaborar para modelar un algoritmo combinatorio eficiente sin recurrir a simulaciones iterativas de fuerza

bruta.

Desarrollo del cuerpo del trabajo

1. Modelado Matemático y Relación de Recurrencia

El núcleo del proyecto consiste en abstraer un tablero de dimensiones $M \times N$ a un modelo matemático de grafos. Si posicionamos el origen en la coordenada $(0,0)$ y el destino en (x,y) , y restringimos los vectores de movimiento a incrementos positivos en un solo eje a la vez (derecha o abajo), el cálculo de trayectorias deja de ser un problema geométrico para convertirse en una función de recurrencia.

La cantidad de caminos únicos para alcanzar cualquier nodo (x,y) está dictaminada por la suma exacta de los caminos posibles para llegar a sus nodos adyacentes predecesores. Esto se formaliza mediante la siguiente ecuación recursiva:

$$f(x,y) = f(x-1,y) + f(x,y-1) \quad (1)$$

Esta formulación define nuestras condiciones de frontera: siempre que $x = 0$ o $y = 0$, el resultado es obligatoriamente 1, ya que existe una única línea recta en los bordes del tablero.

2. Implementación Lógica en el Entorno Zilly+

La resolución algorítmica se llevó a cabo utilizando Zilly+, un lenguaje funcional de propósito académico inspirado en la lógica matemática. Durante el desarrollo, se identificó una limitación sintáctica crítica en el compilador web (ZenSheet): el lenguaje no procesa nativamente los operadores de igualdad ($==$) ni de desigualdad ($>$).

Para sortear este obstáculo y mantener la integridad del flujo recursivo, se rediseñó la condición de parada del algoritmo utilizando exclusivamente el operador relacional de "menor que" ($<$). La instrucción condicional evalúa si el índice posicional es menor a 1 (lo que matemáticamente equivale a que sea cero, dado que trabajamos con números enteros positivos). La función pura implementada quedó definida como:

$$rutas(x,y) \rightarrow if(x < 1, 1, if(y < 1, 1, rutas(x-1,y) + rutas(x,y-1))) \quad (2)$$

Esta estructura garantiza la inmutabilidad de los datos y evita el desbordamiento de la pila de llamadas, demostrando que un entorno con recursos limitados puede resolver problemas combinatorios de crecimiento exponencial mediante lógica deductiva.

3. Arquitectura de Interfaz y Modularidad en C++

Para cumplir con los estándares de ingeniería de software, se aisló el motor recursivo de Zilly+ y se construyó una interfaz interactiva en C++. Se adoptó un paradigma de programación orientada a objetos para encapsular la lógica del entorno bidimensional.

El sistema se divide en tres componentes arquitectónicos:

- **Archivo de Cabecera (Tablero.h):** Define la clase abstracta, privatizando las dimensiones de la matriz para proteger la integridad del cálculo y exponiendo públicamente los métodos de dibujo y cálculo.
- **Implementación (Tablero.cpp):** Contiene la lógica operativa. Incluye un motor de renderizado básico que mapea las coordenadas en arte ASCII para proveer retroalimentación visual al usuario, y replica la fórmula recursiva en tipado estático (`long long`) para procesar tableros de gran escala sin pérdida de precisión.
- **Controlador Principal (main.cpp):** Gestiona el flujo del programa, validando las entradas del usuario para evitar dimensiones nulas o negativas.

4. Análisis de Complejidad Computacional

El algoritmo actual, basado en recursión pura sin memorización, exhibe una complejidad temporal exponencial de orden $\mathcal{O}(2^{x+y})$. Cada llamada a la función genera dos sub-ramas independientes, lo que implica que el procesador recalcula repetidamente el número de rutas para casillas que ya han sido evaluadas en otras ramas del árbol de ejecución. Aunque esta aproximación demuestra la belleza de la programación declarativa, expone los límites del rendimiento en arquitecturas de hardware reales al escalar las dimensiones de la matriz.

Conclusiones y recomendaciones

Conclusiones:

1. Se comprobó empíricamente que la recursividad espacial es un mecanismo superior y más legible para la resolución de problemas de rutas en cuadrículas, eliminando la necesidad de variables de estado complejas y bucles anidados.
2. El lenguaje Zilly+, a pesar de sus restricciones de operadores lógicos, posee la solidez matemática necesaria para estructurar condicionales combinatorios complejos utilizando únicamente operadores relacionales estrictos.

3. La separación de responsabilidades entre el lenguaje matemático puro y un lenguaje modular de producción como C++ resulta ser una metodología de desarrollo de software altamente efectiva.

Recomendaciones:

1. **Implementación de Memorización (Caching):** Para futuras iteraciones del proyecto, se recomienda aplicar técnicas de Programación Dinámica como la "memorización". Al almacenar los resultados en una matriz de memoria, la complejidad temporal se reduciría drásticamente de exponencial $\mathcal{O}(2^{x+y})$ a polinomial $\mathcal{O}(x \cdot y)$.
2. **Obstáculos Aleatorios:** Se sugiere expandir la clase `Tablero` en C++ para incluir casillas nulas o "bloqueadas", acercando el modelo a aplicaciones reales como los algoritmos de navegación GPS.

Bibliografía

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- Rosen, K. H. (2012). *Discrete Mathematics and Its Applications* (7th ed.). McGraw-Hill.
- Alda, E., & Pinto, D. (2026). *Apuntes y Documentación Técnica del Curso CI-2125: Zilly+ y Recursión Funcional*. Universidad Simón Bolívar.

Anexos

Anexo A: Presentación Audiovisual (PDF Adjunto)

Se incluye como archivo independiente el documento `Presentacion_Proyecto_DiegoUgas.pdf`, el cual contiene las diapositivas de soporte visual del proyecto y debe ser evaluado conjuntamente con este informe.

Anexo B: Listados de Código Fuente

B.1: Código Lógico en Zilly+ (rutas.zym)

```
1 ::zilly+
2
3 // Proyecto Final: Calculadora de Rutas en Tablero Reducido
4 // Funcion matematica recursiva: f(x,y) = f(x-1, y) + f(x, y-1)
5
6 Z => Z => Z rutas := fn(Z x) -> fn(Z y) ->
7     if(x < 1, 1,
8         if(y < 1, 1,
9             rutas(x - 1)(y) + rutas(x)(y - 1)
10        )
11    );
12
13 // Pruebas (indices empiezan en 0, un tablero 4x4 llega hasta el indice
14    3)
15 rutas(3)(3)
16 rutas(2)(2)
17 rutas(1)(4)
```

B.2: Interfaz C++ - Cabecera (Tablero.h)

```
1 #ifndef TABLERO_H
2 #define TABLERO_H
3 class Tablero {
4 private:
5     int filas;
6     int columnas;
7 public:
8     Tablero(int f, int c);
9     void dibujar();
10    long long calcularRutas(int x, int y);
11 };
12 #endif
```

B.3: Interfaz C++ - Implementación (Tablero.cpp)

```
1 #include "Tablero.h"
2 #include <iostream>
3
4 Tablero::Tablero(int f, int c) : filas(f), columnas(c) {}
5
6 void Tablero::dibujar() {
7     std::cout << "\nTablero " << filas << "x" << columnas << ":\n";
8     for (int i = 0; i < filas; ++i) {
9         for (int j = 0; j < columnas; ++j) {
10             if (i == 0 && j == 0) std::cout << "[ R ] ";
11             else if (i == filas - 1 && j == columnas - 1) std::cout <<
12                 "[ X ] ";
13             else std::cout << "[ . ] ";
14         }
15         std::cout << "\n";
16     }
17     std::cout << "\n";
18 }
19
20 long long Tablero::calcularRutas(int x, int y) {
21     if (x == 0 || y == 0) return 1;
22     return calcularRutas(x - 1, y) + calcularRutas(x, y - 1);
23 }
```

B.4: Interfaz C++ - Programa Principal (main.cpp)

```
1 #include "Tablero.h"
2 #include <iostream>
3
4 int main() {
5     int f, c;
6     std::cout << "--- Calculadora de Rutas Combinatorias ---\n";
7     std::cout << "Ingrese el numero de filas: ";
8     std::cin >> f;
9     std::cout << "Ingrese el numero de columnas: ";
10    std::cin >> c;
11
12    if (f < 1 || c < 1) {
13        std::cout << "Las dimensiones deben ser mayores a 0.\n";
14        return 1;
15    }
16
17    Tablero t(f, c);
18    t.dibujar();
19 }
```

```
19
20     long long rutas = t.calcularRutas(f - 1, c - 1);
21     std::cout << "Numero de rutas unicas posibles para la pieza: " <<
rutas << "\n";
22     std::cout << "(Resolucion basada en teoria de grafos)\n";
23
24     return 0;
25 }
```